

Arrays in C - Complete Lecture Series: Basics, Functions, Multidimensional Arrays

C - How to Program, 9th edition

Umer Shahid

Department of Electrical Engineering,
UET Lahore

October 27, 2025

Overview

1. Introduction to Arrays
2. Array Basics
3. Initializing Arrays
4. Array Processing Examples
5. Character Arrays and Strings
6. Passing Arrays to Functions
7. Sorting and Searching
8. Two-Dimensional Arrays
9. Solved Problems
10. Common Mistakes and Pitfalls
11. Practice Exercises
12. Summary and Key Takeaways
13. Additional Solved Problems

What is an Array?

Definition

An **array** is a group of elements of the **same type** stored **contiguously** (one after another) in memory.

Think of it like:

- A row of mailboxes in an apartment building
- A parking lot with numbered spaces
- A row of lockers in school

Key Points

- All elements must be of the **same data type**
- Elements are stored **consecutively** in memory
- Arrays are **static** - size is fixed at creation

Why Do We Need Arrays?

Problem Without Arrays:

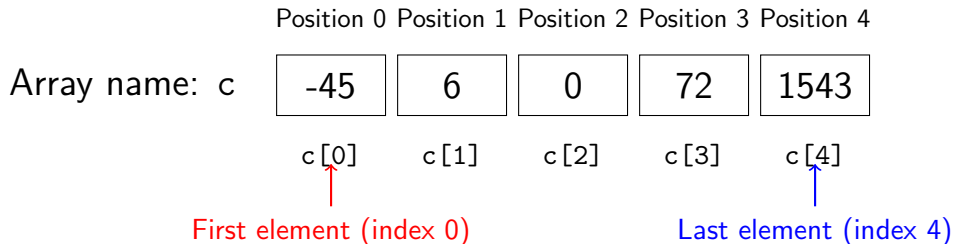
```
1 // Storing 5 student grades without arrays
2 int grade1, grade2, grade3, grade4, grade5;
3 grade1 = 85;
4 grade2 = 90;
5 grade3 = 78;
6 grade4 = 92;
7 grade5 = 88;
```

Solution With Arrays:

```
1 // Storing 5 student grades with array
2 int grades[5];
3 grades[0] = 85;
4 grades[1] = 90;
5 grades[2] = 78;
6 grades[3] = 92;
7 grades[4] = 88;
```

Imagine handling 100 or 1000 grades!

Understanding Array Structure



Important!

- Array indexing starts at **0**, not 1
- For an array of size n , valid indices are 0 to $n - 1$
- Index is also called **subscript**

Defining Arrays

Syntax:

```
dataType arrayName[arraySize];
```

Examples:

```
1 int c[5];           // Array of 5 integers
2 double grades[100]; // Array of 100 doubles
3 char name[20];      // Array of 20 characters
4 float temps[7];     // Array of 7 floats
```

What Happens in Memory?

When you write `int c[5];`, the computer:

1. Allocates space for 5 integers ($5 \times 4 \text{ bytes} = 20 \text{ bytes}$)
2. Reserves consecutive memory locations
3. Does NOT initialize values (contains garbage)

Accessing Array Elements

Syntax:

arrayName[index]

```
1 int c[5];
2 // Assigning values
3 c[0] = 10;
4 c[2] = 30;
5 c[3] = 40;
6 c[4] = 50;
7 // Uses
8 printf("%d", c[0]);           // Prints: 10
9 int x = c[2] + c[3];          // x = 30 + 40 = 70
10 c[1] = c[0] + c[4];           // c[1] = 10 + 50 = 60
```

Common Mistake

c[5] = 100; // ERROR! Valid indices: 0-4 only

Self-Check Question 1

Question

Which of the following statements is **false**?

- a) Any array element may be referred to by giving the array's name followed by the element's position number in square brackets ([]).
- b) Every array's first element has position number 1.
- c) An array name can contain only letters, digits and underscores and cannot begin with a digit.
- d) The position number in square brackets is called the element's subscript (or index).

Self-Check Question 1

Question

Which of the following statements is **false**?

- a) Any array element may be referred to by giving the array's name followed by the element's position number in square brackets ([]).
- b) Every array's first element has position number 1.
- c) An array name can contain only letters, digits and underscores and cannot begin with a digit.
- d) The position number in square brackets is called the element's subscript (or index).

Answer

b) is false. Every array's first element has position number **0**, not 1!

Method 1: Using Loops

```
1 #include <stdio.h>
2 #define SIZE 5
3
4 int main(void) {
5     int n[SIZE]; // Declare array
6
7     // Initialize all elements to 0
8     for (int i = 0; i < SIZE; ++i) {
9         n[i] = 0;
10    }
11
12    // Print array
13    printf("Element\tValue\n");
14    for (int i = 0; i < SIZE; ++i) {
15        printf("%zu\t%d\n", i, n[i]);
16    }
17
18    return 0;
19 }
```

Method 2: Initializer List

```
1 // Initialize with specific values
2 int n[5] = {32, 27, 64, 18, 95};
3
4 // Partially initialize (rest become 0)
5 int m[5] = {10, 20}; // m[0]=10, m[1]=20, m[2]=0, m[3]=0, m[4]=0
6
7 // Quick way to initialize all to zero
8 int p[5] = {0}; // All elements become 0
9
10 // Let compiler count elements
11 int q[] = {1, 2, 3, 4, 5}; // Size automatically 5
```

Error

```
int x[3] = {1, 2, 3, 4}; // ERROR! Too many initializers
```

Using Symbolic Constants

```
1 #include <stdio.h>
2 #define SIZE 5    // Symbolic constant
3
4 int main(void) {
5     int s[SIZE] = {0};
6
7     // Fill array with even numbers: 2, 4, 6, 8, 10
8     for (int j = 0; j < SIZE; ++j) {
9         s[j] = 2 + 2 * j;
10    }
11
12    // Print array
13    for (int j = 0; j < SIZE; ++j) {
14        printf("s[%zu] = %d\n", j, s[j]);
15    }
16
17    return 0;
18 }
```

Self-Check Question 2

Question

Write a statement that displays the `int` product of the values contained in the first four elements of `int` array `grades`.

Self-Check Question 2

Question

Write a statement that displays the `int` product of the values contained in the first four elements of `int` array `grades`.

Answer

```
1 printf("%d", grades[0] * grades[1] * grades[2] * grades[3]);
```

Explanation:

- First four elements have indices: 0, 1, 2, 3
- Use multiplication operator (*)
- Array indices go inside square brackets

Example 1: Summing Array Elements

```
1 #include <stdio.h>
2 #define SIZE 5
3
4 int main(void) {
5     int a[SIZE] = {1, 2, 3, 4, 5};
6     int total = 0;
7
8     // Sum all elements
9     for (int i = 0; i < SIZE; ++i) {
10         total += a[i]; // total = total + a[i]
11     }
12
13     printf("The total is %d\n", total);
14     // Output: The total is 15
15
16     return 0;
17 }
```

Step-by-step: $\text{total} = 0 + 1 + 2 + 3 + 4 + 5 = 15$

Example 2: Finding Maximum

```
1 #include <stdio.h>
2 #define SIZE 5
3
4 int main(void) {
5     int numbers[SIZE] = {45, 23, 89, 12, 67};
6     int max = numbers[0]; // Assume first is maximum
7
8     // Check remaining elements
9     for (int i = 1; i < SIZE; ++i) {
10         if (numbers[i] > max) {
11             max = numbers[i];
12         }
13     }
14
15     printf("Maximum value: %d\n", max);
16     // Output: Maximum value: 89
17
18     return 0;
19 }
```

Example 3: Histogram/Bar Chart

```
1 #include <stdio.h>
2 #define SIZE 5
3
4 int main(void) {
5     int n[SIZE] = {19, 3, 15, 7, 11};
6
7     printf("Element\tValue\tBar Chart\n");
8
9     for (int i = 0; i < SIZE; ++i) {
10         printf("%zu\t%d\t", i, n[i]);
11
12         // Print asterisks
13         for (int j = 1; j <= n[i]; ++j) {
14             printf("*");
15         }
16         printf("\n");
17     }
18     return 0;
19 }
```

Example 3 Output

Element	Value	Bar Chart
0	19	*****
1	3	***
2	15	*****
3	7	*****
4	11	*****

How it works:

- Outer loop goes through each array element
- Inner loop prints asterisks (as many as the value)
- For `n[0] = 19`, inner loop prints 19 asterisks

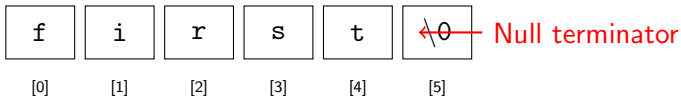
Character Arrays for Strings

Important Concept

In C, a **string** is a character array ending with '`\0`' (null character).

Two ways to initialize:

```
1 // Method 1: String literal
2 char string1[] = "first";
3
4 // Method 2: Character list
5 char string2[] = {'f', 'i', 'r', 's', 't', '\0'};
```



Size = 6 (5 characters + 1 null terminator)

Reading and Printing Strings

```
1 #include <stdio.h>
2 #define SIZE 20
3
4 int main(void) {
5     char name[SIZE];
6
7     // Reading string (no & needed!)
8     printf("Enter your name: ");
9     scanf("%19s", name); // Read max 19 chars
10    // Printing string
11    printf("Hello, %s!\n", name);
12    return 0;
13 }
```

Security Note

Always use field width (%19s) to prevent buffer overflow! For a 20-character array, read at most 19 characters (leaving room for '\0').

Accessing Individual Characters

```
1 #include <stdio.h>
2
3 int main(void) {
4     char word[] = "HELLO";
5
6     // Print each character
7     printf("Individual characters:\n");
8     for (int i = 0; i < 5; ++i) {
9         printf("%c ", word[i]);
10    }
11    printf("\n");
12    // Output: H E L L O
13
14    // Modify a character
15    word[0] = 'J';
16    printf("%s\n", word); // Output: JELLO
17
18    return 0;
19 }
```

How to Pass Arrays

Arrays are passed by reference automatically!

```
1 #include <stdio.h>
2 #define SIZE 5
3
4 void printArray(int arr[], int size);
5
6 int main(void) {
7     int numbers[SIZE] = {10, 20, 30, 40, 50};
8
9     // Pass array (no brackets needed)
10    printArray(numbers, SIZE);
11
12    return 0;
13 }
14
15 void printArray(int arr[], int size) {
16     for (int i = 0; i < size; ++i) {
17         printf("%d ", arr[i]);
18     }
```

Pass by Reference vs Pass by Value

Entire Array (by reference):

```
1 void modifyArray(int a[], int size)
  {
2     a[0] = 100;
3 }
4
5 int main(void) {
6     int arr[3] = {1,2,3};
7     modifyArray(arr, 3);
8     // arr[0] is now 100!
9 }
```

Individual Element (by value):

```
1 void modifyElement(int x) {
2     x = 100;
3 }
4
5 int main(void) {
6     int arr[3] = {1,2,3};
7     modifyElement(arr[0]);
8     // arr[0] is still 1!
9 }
```

Key Point

Passing entire array: changes affect original

Passing single element: changes don't affect original

Using const to Protect Arrays

```
1 #include <stdio.h>
2 // Function CANNOT modify array
3 void printArray(const int arr[], int size) {
4     for (int i = 0; i < size; ++i) {
5         printf("%d ", arr[i]);
6         // arr[i] = 0; // ERROR! Cannot modify
7     }
8 }
9 int main(void) {
10     int data[5] = {1, 2, 3, 4, 5};
11     printArray(data, 5);
12     return 0;
13 }
```

Benefits of const:

- Prevents accidental modifications
- Makes code safer and clearer
- Documents that function only reads the array

Bubble Sort - The Concept

How Bubble Sort Works

1. Compare adjacent pairs of elements
2. If they're in wrong order, swap them
3. Repeat until array is sorted
4. Larger values "bubble" to the end

Example: Sorting [5, 2, 8, 1]

Pass 1:	[5, 2, 8, 1]	→ swap 5,2	[2, 5, 8, 1]
	[2, 5, 8, 1]	→ no swap	[2, 5, 8, 1]
	[2, 5, 8, 1]	→ swap 8,1	[2, 5, 1, 8]
Pass 2:	[2, 5, 1, 8]	→ no swap	[2, 5, 1, 8]
	[2, 5, 1, 8]	→ swap 5,1	[2, 1, 5, 8]
Pass 3:	[2, 1, 5, 8]	→ swap 2,1	[1, 2, 5, 8]

Bubble Sort - Implementation

```
1 void bubbleSort(int a[], int size) {  
2     // Outer loop: number of passes  
3     for (int pass = 1; pass < size; ++pass) {  
4  
5         // Inner loop: comparisons per pass  
6         for (int i = 0; i < size - 1; ++i) {  
7  
8             // Compare adjacent elements  
9             if (a[i] > a[i + 1]) {  
10                // Swap them  
11                int temp = a[i];  
12                a[i] = a[i + 1];  
13                a[i + 1] = temp;  
14            }  
15        }  
16    }  
17 }
```

Note: We need a temporary variable for swapping!

Linear Search

How Linear Search Works

1. Start from first element
2. Compare each element with search key
3. If found, return index
4. If not found, return -1

Example: Search for 30 in [10, 25, 30, 45, 50]

Check index 0: 10 $\neq$ 30 Continue

Check index 1: 25 $\neq$ 30 Continue

Check index 2: 30 = 30 Found! Return 2

Performance

Average case: Checks half the elements

Worst case: Checks all elements

Linear Search - Implementation

```
1 int linearSearch(const int arr[], int key, int size) {
2     // Loop through array
3     for (int i = 0; i < size; ++i) {
4         if (arr[i] == key) {
5             return i; // Found! Return index
6         }
7     }
8
9     return -1; // Not found
10 }
11
12 // Using it:
13 int position = linearSearch(myArray, 30, 10);
14 if (position != -1) {
15     printf("Found at index %d\n", position);
16 } else {
17     printf("Not found\n");
18 }
```

Introduction to 2D Arrays

What is a 2D Array?

A table or grid of values with rows and columns.

Syntax:

```
dataType arrayName[rows][columns];
```

Example: `int grades[3][4];` ← 3 rows, 4 columns

[0][0]	[0][1]	[0][2]	[0][3]
[1][0]	[1][1]	[1][2]	[1][3]
[2][0]	[2][1]	[2][2]	[2][3]

Total elements: $3 \times 4 = 12$

Initializing 2D Arrays

```
1 // Method 1: Row by row
2 int matrix[2][3] = {
3     {1, 2, 3},    // Row 0
4     {4, 5, 6}     // Row 1
5 };
6
7 // Method 2: Partial initialization
8 int table[2][3] = {
9     {1, 2},        // Row 0: 1, 2, 0
10    {4}             // Row 1: 4, 0, 0
11 };
12
13 // Method 3: All zeros
14 int zeros[2][3] = {0};
```

Remember

Missing values are automatically initialized to 0!

Processing 2D Arrays

```
1 #include <stdio.h>
2
3 int main(void) {
4     int matrix[2][3] = {
5         {1, 2, 3},
6         {4, 5, 6}
7     };
8
9     // Print all elements
10    for (int row = 0; row < 2; ++row) {
11        for (int col = 0; col < 3; ++col) {
12            printf("%d ", matrix[row][col]);
13        }
14        printf("\n"); // New line after each row
15    }
16
17    return 0;
18 }
```

Output:

Problem 1: Average of Array

Problem: Calculate and print the average of values in an array.

```
1 #include <stdio.h>
2 #define SIZE 5
3
4 int main(void) {
5     int numbers[SIZE] = {10, 20, 30, 40, 50};
6     int sum = 0;
7
8     // Calculate sum
9     for (int i = 0; i < SIZE; ++i) {
10         sum += numbers[i];
11     }
12
13     // Calculate average
14     double average = (double)sum / SIZE;
15
16     printf("Sum: %d\n", sum);
17     printf("Average: %.2f\n", average);
18 }
```

Problem 2: Reverse an Array

Problem: Reverse the elements of an array.

```
1 #include <stdio.h>
2 #define SIZE 5
3
4 int main(void) {
5     int arr[SIZE] = {1, 2, 3, 4, 5};
6
7     // Reverse using swapping
8     for (int i = 0; i < SIZE / 2; ++i) {
9         int temp = arr[i];
10        arr[i] = arr[SIZE - 1 - i];
11        arr[SIZE - 1 - i] = temp;
12    }
13
14    // Print reversed array
15    for (int i = 0; i < SIZE; ++i) {
16        printf("%d ", arr[i]);
17    }
18    printf("\n");
```

Problem 3: Count Occurrences

Problem: Count how many times a number appears in an array.

```
1 #include <stdio.h>
2 #define SIZE 8
3
4 int main(void) {
5     int numbers[SIZE] = {5, 2, 5, 8, 5, 1, 5, 3};
6     int searchValue = 5;
7     int count = 0;
8
9     // Count occurrences
10    for (int i = 0; i < SIZE; ++i) {
11        if (numbers[i] == searchValue) {
12            count++;
13        }
14    }
15
16    printf("%d appears %d times\n", searchValue, count);
17
18    return 0;
```

Problem 5: Find Minimum and Maximum

Problem: Find both minimum and maximum values in an array.

```
1 #include <stdio.h>
2 #define SIZE 6
3 int main(void) {
4     int arr[SIZE] = {45, 23, 89, 12, 67, 34};
5     int min = arr[0];
6     int max = arr[0];
7     for (int i = 1; i < SIZE; ++i) {
8         if (arr[i] < min) {
9             min = arr[i];
10        }
11        if (arr[i] > max) {
12            max = arr[i];
13        }
14    }
15    printf("Minimum: %d\n", min);
16    printf("Maximum: %d\n", max);
17    return 0;
18 }
```

Problem 6: Sum of Each Row (2D Array)

Problem: Calculate sum of each row in a 2D array.

```
1 #include <stdio.h>
2 #define ROWS 3
3 #define COLS 4
4
5 int main(void) {
6     int matrix[ROWS][COLS] = {
7         {1, 2, 3, 4},
8         {5, 6, 7, 8},
9         {9, 10, 11, 12}
10    };
11    for (int row = 0; row < ROWS; ++row) {
12        int sum = 0;
13        for (int col = 0; col < COLS; ++col) {
14            sum += matrix[row][col];
15        }
16        printf("Sum of row %zu: %d\n", row, sum);
17    }
18    return 0;
```

Problem 7: Check if Array is Sorted

Problem: Check if an array is sorted in ascending order.

```
1 #include <stdio.h>
2 #define SIZE 5
3 int main(void) {
4     int arr[SIZE] = {10, 20, 30, 40, 50};
5     int isSorted = 1; // Assume sorted (true)
6     for (int i = 0; i < SIZE - 1; ++i) {
7         if (arr[i] > arr[i + 1]) {
8             isSorted = 0; // Not sorted (false)
9             break;
10        }
11    }
12    if (isSorted) {
13        printf("Array is sorted\n");
14    } else {
15        printf("Array is not sorted\n");
16    }
17    return 0;
18 }
```

Problem 8: Frequency Counter

Problem: Count frequency of numbers 1-5 in an array.

```
1 #include <stdio.h>
2 #define SIZE 10
3 #define FREQ_SIZE 6 // Index 0 unused, 1-5 for values
4
5 int main(void) {
6     int data[SIZE] = {3, 1, 4, 1, 5, 2, 3, 3, 1, 2};
7     int frequency[FREQ_SIZE] = {0};
8
9     // Count frequencies
10    for (int i = 0; i < SIZE; ++i) {
11        ++frequency[data[i]];
12    }
13
14    // Print results
15    printf("Value\tFrequency\n");
16    for (int i = 1; i < FREQ_SIZE; ++i) {
17        printf("%zu\t%d\n", i, frequency[i]);
18    }
```

Common Mistake 1: Array Bounds

Wrong Code

```
1 int arr[5];  
2 for (int i = 0; i <= 5; i++) { // ERROR!  
3     arr[i] = 0;  
4 }
```

Correct Code

```
1 int arr[5];  
2 for (int i = 0; i < 5; i++) { // Correct  
3     arr[i] = 0;  
4 }
```

Problem: Array of size 5 has indices 0-4, not 0-5!

Accessing arr[5] causes: Buffer overflow, program crash, or undefined behavior

Common Mistake 2: Uninitialized Arrays

Dangerous Code

```
1 int numbers[5];  
2 printf("%d\n", numbers[0]); // Prints garbage!
```

Safe Code

```
1 int numbers[5] = {0}; // Initialize to zero  
2 printf("%d\n", numbers[0]); // Prints 0
```

Lesson: Always initialize arrays before using them!

Common Mistake 3: String Without Null Terminator

Wrong Code

```
1 char name[5] = {'J', 'o', 'h', 'n'};
2 printf("%s\n", name); // May print garbage!
```

Correct Code

```
1 char name[5] = {'J', 'o', 'h', 'n', '\0'};
2 printf("%s\n", name); // Prints: John
3
4 // OR better:
5 char name[] = "John"; // Automatic null terminator
```

Remember: Strings need `'\0'` at the end!

Common Mistake 4: Comparing Arrays Incorrectly

Wrong Code

```
1 int a[3] = {1, 2, 3};
2 int b[3] = {1, 2, 3};
3
4 if (a == b) { // ERROR! Compares addresses, not contents
5     printf("Equal\n");
6 }
```

Correct Code

```
1 int a[3] = {1, 2, 3};
2 int b[3] = {1, 2, 3};
3 int equal = 1;
4
5 for (int i = 0; i < 3; ++i) {
6     if (a[i] != b[i]) {
7         equal = 0;
8         break;
9     }
10 }
```

Common Mistake 5: scanf Without Field Width

Dangerous Code

```
1 char name[10];  
2 scanf("%s", name); // User could type 50 characters!
```

Safe Code

```
1 char name[10];  
2 scanf("%9s", name); // Read max 9 chars (+ null)
```

Security Issue: Buffer overflow can crash program or create security vulnerabilities!

Exercise 1: Basic Array Operations

Task: Write a program that:

1. Creates an array of 10 integers
2. Fills it with numbers 1 through 10
3. Prints all elements
4. Calculates and prints the sum

Expected Output:

Array: 1 2 3 4 5 6 7 8 9 10

Sum: 55

Hints:

- Use a loop to fill the array: `arr[i] = i + 1`
- Use another loop to calculate sum

Exercise 2: Even and Odd Counter

Task: Write a program that:

1. Takes an array: {12, 7, 23, 8, 15, 20, 9, 14}
2. Counts how many even numbers
3. Counts how many odd numbers
4. Prints both counts

Expected Output:

Even numbers: 4

Odd numbers: 4

Hint: Use `arr[i] % 2 == 0` to check if even

Exercise 3: Temperature Converter

Task: Write a program that:

1. Stores 7 temperatures in Celsius: {0, 10, 20, 25, 30, 35, 40}
2. Converts each to Fahrenheit: $F = C \times 9/5 + 32$
3. Stores Fahrenheit values in another array
4. Prints both arrays side by side

Expected Output:

Celsius	Fahrenheit
0	32
10	50
...	

Exercise 4: Array Rotation

Task: Rotate array elements one position to the right.

Example:

- Input: [1, 2, 3, 4, 5]
- Output: [5, 1, 2, 3, 4]

Algorithm:

1. Store last element in temporary variable
2. Shift all elements one position right
3. Place temporary variable at position 0

Challenge: Can you rotate left instead of right?

Exercise 5: Matrix Addition

Task: Add two 2×2 matrices.

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Hint:

```
1 for (row = 0; row < 2; row++) {  
2     for (col = 0; col < 2; col++) {  
3         result[row][col] = matrix1[row][col]  
4                               + matrix2[row][col];  
5     }  
6 }
```

Key Concepts Summary

1. **Arrays** store multiple values of the same type
2. **Indexing** starts at 0, not 1
3. **Size** is fixed at creation time
4. **Initialization** can use initializer lists or loops
5. **Bounds checking** is your responsibility!
6. **Strings** are character arrays ending with `'\0'`
7. **Functions** receive arrays by reference
8. **2D arrays** use two indices: `[row][column]`

Important Reminders

Security

- Always check array bounds
- Use field width with `scanf`: `%19s` for 20-char array
- Initialize arrays before use
- Remember null terminator for strings

Best Practices

- Use `#define` for array sizes
- Use `size_t` for array indices
- Use `const` when function shouldn't modify array
- Comment your code clearly

Array vs Individual Variables

Without Arrays	With Arrays
<code>int grade1, grade2, ...</code> 100 separate variables Difficult to process Can't pass to function easily Hard to maintain	<code>int grades[100];</code> One array variable Easy loops Pass entire array Easy to maintain

Conclusion

Arrays make programs:

- More organized
- Easier to maintain
- More efficient
- Scalable

Common Array Algorithms - Quick Reference

Operation	Key Code
Initialize to zero	<code>int arr[10] = {0};</code>
Sum elements	<code>sum += arr[i];</code>
Find maximum	<code>if (arr[i] > max) max = arr[i];</code>
Find minimum	<code>if (arr[i] < min) min = arr[i];</code>
Search	<code>if (arr[i] == key) return i;</code>
Swap elements	<code>temp = a; a = b; b = temp;</code>
Copy array	<code>dest[i] = source[i];</code>
Reverse	<code>swap(arr[i], arr[n-1-i]);</code>
Count occurrences	<code>if (arr[i] == x) count++;</code>

What's Next?

In upcoming lectures, you'll learn:

- **Pointers** - Understanding memory addresses
- **Dynamic Memory** - Creating arrays at runtime
- **Advanced Sorting** - QuickSort, MergeSort
- **String Functions** - strlen, strcpy, strcmp
- **Multidimensional Arrays** - 3D arrays and beyond
- **Structures** - Combining different data types

Practice is key! Work on:

- End-of-chapter exercises (6.10-6.15)
- Write programs to manipulate arrays
- Debug common errors

Tips for Success

1. **Draw diagrams** - Visualize array in memory
2. **Use paper** - Trace through loops manually
3. **Start simple** - Begin with small arrays (size 5)
4. **Test boundaries** - Check first and last elements
5. **Print often** - Use printf to debug
6. **Read error messages** - They tell you what's wrong
7. **Practice daily** - Write code every day
8. **Ask questions** - No question is too small

Remember

Programming is a skill developed through practice, not just theory!

Quick Quiz

1. What is the index of the first element in an array?

Quick Quiz

1. What is the index of the first element in an array?
2. How do you initialize all elements of an array to zero?

Quick Quiz

1. What is the index of the first element in an array?
2. How do you initialize all elements of an array to zero?
3. Can you change an array's size after creation?

Quick Quiz

1. What is the index of the first element in an array?
2. How do you initialize all elements of an array to zero?
3. Can you change an array's size after creation?
4. What happens if you access `arr[10]` in a 10-element array?

Quick Quiz

1. What is the index of the first element in an array?
2. How do you initialize all elements of an array to zero?
3. Can you change an array's size after creation?
4. What happens if you access `arr[10]` in a 10-element array?
5. How are arrays passed to functions - by value or reference?

Answers:

1. 0 (zero)
2. `int arr[10] = {0};`
3. No, arrays have fixed size
4. Undefined behavior (accessing out of bounds)
5. By reference (automatically)

Problem 9: Remove Duplicates (Simple)

Problem: Print unique elements from an array.

```
1 #include <stdio.h>
2 #define SIZE 8
3 int main(void) {
4     int arr[SIZE] = {1, 2, 2, 3, 4, 4, 5, 5};
5     printf("Unique elements: ");
6     for (int i = 0; i < SIZE; ++i) {
7         int isDuplicate = 0;
8         for (int j = 0; j < i; ++j) {
9             if (arr[i] == arr[j]) {
10                 isDuplicate = 1;
11                 break;}}
12         if (!isDuplicate) {
13             printf("%d ", arr[i]);}}
14     return 0;
15 }
```

Output: Unique elements: 1 2 3 4 5

Problem 10: Second Largest Element

Problem: Find the second largest element in an array.

```
1 #include <stdio.h>
2 #define SIZE 6
3
4 int main(void) {
5     int arr[SIZE] = {45, 23, 89, 12, 67, 34};
6     int largest = arr[0];
7     int secondLargest = arr[0];
8     for (int i = 1; i < SIZE; ++i) {
9         if (arr[i] > largest) {
10             secondLargest = largest;
11             largest = arr[i];
12         } else if (arr[i] > secondLargest && arr[i] != largest) {
13             secondLargest = arr[i];
14         }
15     }
16     printf("Second largest: %d\n", secondLargest);
17     return 0;}
```

Output: Second largest: 67

Problem 11: Array Merge

Problem: Merge two sorted arrays into one sorted array.

```
1 #include <stdio.h>
2
3 int main(void) {
4     int a[] = {1, 3, 5};
5     int b[] = {2, 4, 6};
6     int c[6];
7     int i = 0, j = 0, k = 0;
8
9     // Merge
10    while (i < 3 && j < 3) {
11        if (a[i] < b[j]) {
12            c[k++] = a[i++];
13        } else {
14            c[k++] = b[j++];
15        }
16    }
17    while (i < 3) c[k++] = a[i++];
18    while (j < 3) c[k++] = b[j++];
```

Problem 12: Transpose of Matrix

Problem: Find transpose of a 2×3 matrix.

```
1  #include <stdio.h>
2
3  int main(void) {
4      int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};
5      int transpose[3][2];
6
7      // Calculate transpose
8      for (int i = 0; i < 2; ++i) {
9          for (int j = 0; j < 3; ++j) {
10             transpose[j][i] = matrix[i][j];
11         }
12     }
13
14     // Print transpose
15     for (int i = 0; i < 3; ++i) {
16         for (int j = 0; j < 2; ++j) {
17             printf("%d ", transpose[i][j]);
18         }
```

Thank You!

See you next week!

Keep practicing and happy coding!